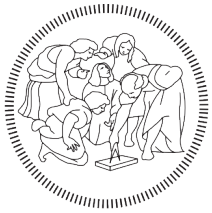
Artificial Neural Networks and Deep Learning - Challenge 1
Image classification

November 2022

Riccardo Luigi Aielli
Salvatore Buono
Simone Buranti



POLITECNICO
MILANO 1863

Contents

1 Introduction **2**

2 First Approach **2**
2.1 Dataset split 2
2.2 Base CNN 2
2.3 Increasing performance with data augmentation 2

3 Transfer Learning and Fine Tuning **2**
3.1 VGG16 3
3.2 DenseNet201 3
3.2.1 K-fold Cross Validation 3
3.3 ConvNeXt Base 3
3.3.1 CutMix 4

4 Final Results **4**

1 Introduction

The goal of this project is to build a Convolutional Neural Network able to classify plant images into 8 classes. In this report, we illustrate the process that brought us to a 91% accuracy score on the test set. Our work has been performed on Google Colab, as it offers a convenient setup and free computational units. Throughout the challenge, we adopted a bottom-up approach starting from simpler models and progressively increasing the pipeline complexity.

2 First Approach

2.1 Dataset split

We split the dataset into training and validation sets (85/15) using stratified sampling, so that both sets preserve the class proportions of the original data. Moreover, due to the scarcity of data, we did not hold out a local test set and instead evaluated test performance directly on the *Development* test set provided on CodaLab.

2.2 Base CNN

As a first step, we built a CNN from scratch, closely based on the architecture presented in class, to establish a baseline against which all subsequent models could be compared.

2.3 Increasing performance with data augmentation

To tackle data scarcity, we adopted data augmentation techniques that, by introducing slightly modified or synthetic data, allow us to both increase and enrich the data samples. Beyond increasing the amount of data, augmentation acts as a regularizer as well, reducing overfitting. As for preprocessing, we normalized the input images through the `rescale` argument of the `ImageDataGenerator` class. Figure 1 shows the performances obtained at this phase.

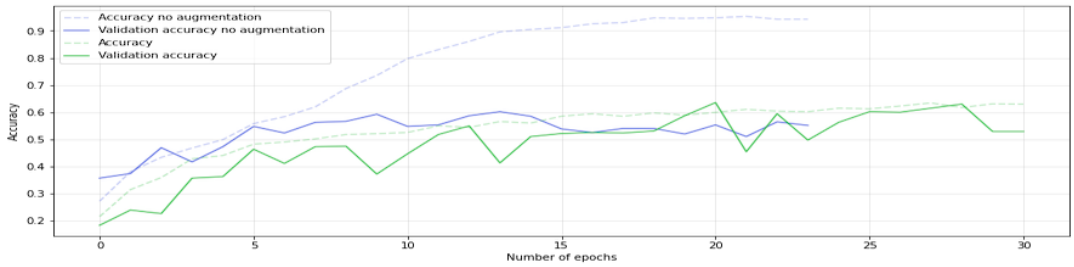


Figure 1: Accuracy of base model with data augmentation

3 Transfer Learning and Fine Tuning

Not satisfied with the performance of the handmade model, we decided to use the Transfer Learning and Fine Tuning techniques. In order to improve model accuracy, we decided to duplicate the dataset in order to obtain different types of augmentation of the same image and to insert a `GlobalAveragePooling` layer with the aim of reducing overfitting and enforcing correspondences between feature maps and categories.

We first tried VGG16, DenseNet121, DenseNet201, then InceptionV3, Xception, and finally ConvNeXt Base. Here we present the most relevant models.

3.1 VGG16

We started with VGG16, presented during the exercise sessions, as a simple and well-established architecture to benchmark Transfer Learning on our dataset. Figure 2 shows the results obtained with Transfer Learning and Fine Tuning, freezing the first 14 layers.

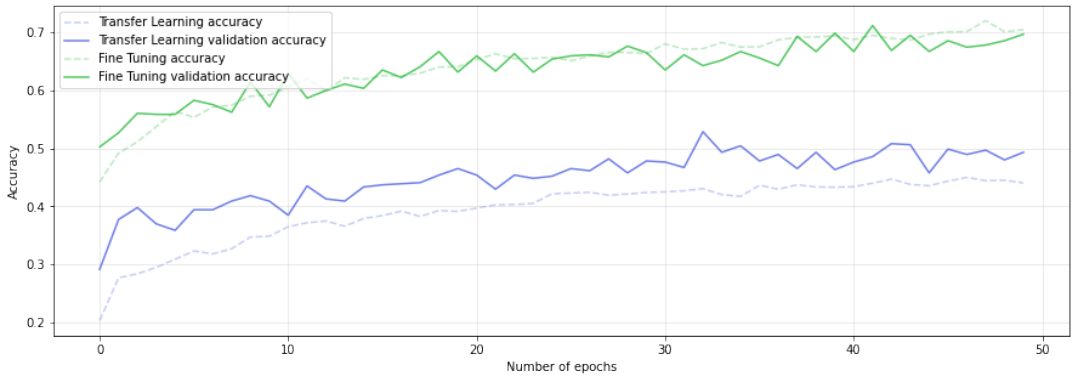


Figure 2: Accuracy of the VGG16 model

3.2 DenseNet201

We then moved to DenseNet201, which substantially improved performance over VGG16. Nevertheless, we explored further techniques and architectures.

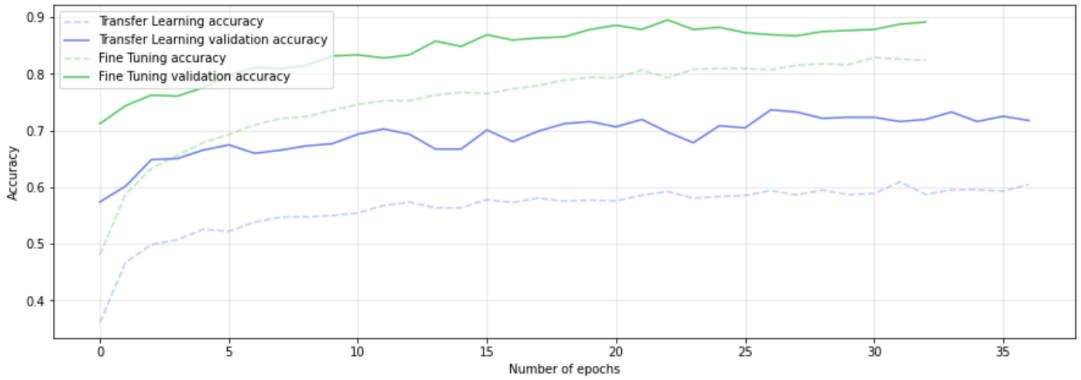


Figure 3: Accuracy of DenseNet201 model

3.2.1 K-fold Cross Validation

While tuning the hyperparameters, we also applied 10-fold cross-validation to obtain a more reliable estimate of the model's generalization performance.

3.3 ConvNeXt Base

By taking as a reference the Keras-Applications documentation, we finally tried to implement Transfer Learning and Fine Tuning with this last CNN family, which showed

promising statistics. At the beginning, we tried ConvNeXtSmall, already noticing a significant improvement in the accuracy on our dataset. Even better performances were reached when we imported the following, slightly more complex, ConvNeXtBase. Given the prohibitive size and computational resources demanded by ConvNeXtLarge, we decided to focus on the previous one.

3.3.1 CutMix

In order to further improve our data augmentation pipeline, we decided to adopt CutMix regularization strategy. In particular, we instantiated two `ImageDataGenerator` linked to the same training folder, and with the same data augmentation settings. Then we replaced the previous one with a CutMix version from `cutmix_keras`, fed with the two simple instances.

4 Final Results

The model obtained as previously described reached high values of accuracy on our training and validation sets. However, by submitting it to the platform, we were still witnessing a performance gap of about 5% accuracy with respect to our local scores, a sign of overfitting. Thus, we applied three additional changes. Firstly, to further reduce overfitting, we introduced a `GaussianNoise` layer after the input layer of the CNN architecture. This, combined with the CutMix regularization strategy, allowed us to significantly reduce the drawbacks of having a limited dataset and to improve the model's ability to generalize to unseen images. Secondly, we added two `Dense` layers followed by a `Dropout` layer in the final part of the architecture, in order to improve the effectiveness of the classification module. Finally, still to enhance the generalization of the model, we reduced the patience of the Early Stopping from 10 to 7, as we noticed that sparse and marginal validation accuracy peaks kept resetting the patience counter, unnecessarily lengthening the training.

These ultimate refinements allowed us to reach an overall accuracy of 91.1% on the reference test set (Figure 4).

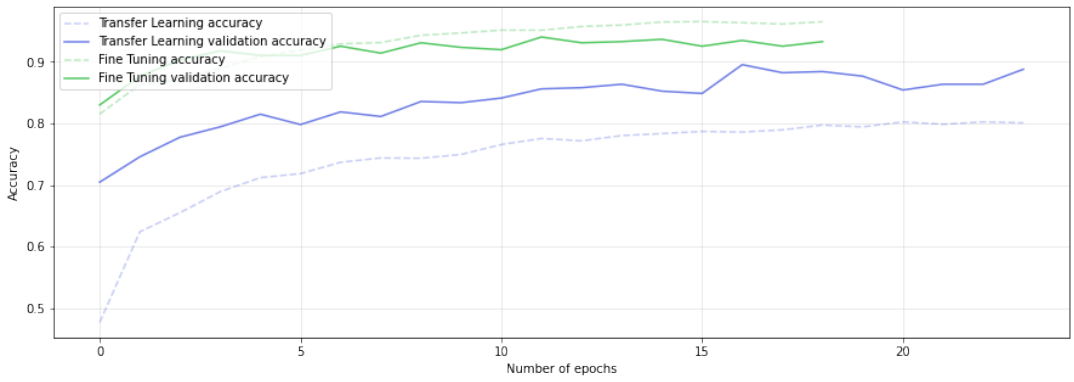


Figure 4: Accuracy of the final model